Data 201

Statistical Methods in Data Science

Linear Regression: R → Python Bridge

Week 4

Dr. Rebin Muhammad

Welcome — Week 4

Weeks 1–3 gave you Python fundamentals, pandas, and visualization. This week we connect to **modeling**: linear regression, the same model you know from R's `lm()`, now in Python with **statsmodels** (inference) and **scikit-learn** (prediction).

Today (75 min): same model, different interfaces; inference vs prediction; fitting and interpreting in both libraries; and choosing the right tool.

Have the **Week 4 Notebook** and your **R→Python cheat sheet** open.

What we'll do today (Week 4)

1. **Where this fits** — Continuous outcomes and linear regression in the course arc
2. **Same model, different interfaces** — R vs Python: defaults, outputs, inference vs prediction
3. **R: lm()** — Formula, coefficients, SEs, p-values
4. **Python: statsmodels** — Inference-focused (like lm); formula interface
5. **Interpretation** — Coefficients as conditional effects; simple vs multiple; categoricals
6. **Python: scikit-learn** — Prediction-focused; train/test; no formula, no inference output
7. **Evaluation and choice** — RMSE, R^2; when to use which tool
8. **Practice** — Explain, compare, decide

Learning goals

By the end of today you will be able to:

1. **Explain** simple and multiple linear regression conceptually
2. **Translate** R `lm()` models into Python using statsmodels
3. **Distinguish** inference-focused vs prediction-focused workflows
4. **Fit** linear regression models for explanation and for prediction
5. **Choose** appropriate evaluation metrics and the right tool for the goal

Where this fits in the course

So far: Python basics (Week 1), data wrangling with pandas (Week 2), visualization (Week 3).

Today: Continuous outcomes and **linear regression** — the R → Python bridge for modeling.

Next: Logistic regression (binary outcomes).

Same model, different interfaces

Key idea: Linear regression is the **same statistical model** in R and Python (same assumptions, same interpretation of coefficients).

Differences are in: defaults, output format, and **workflow** — whether you care mainly about **inference** (effects, uncertainty) or **prediction** (accuracy on new data).

Linear regression in R: lm()

Typical R workflow:

```
lm(price ~ size + bedrooms + neighborhood, data = df)
```

Characteristics: Inference-first, formula interface, coefficients + SEs + p-values out of the box.

$$\text{price} = \beta_0 + \beta_1(\text{size}) + \beta_2(\text{bedrooms}) + \beta_3(\text{neighborhood}) + \varepsilon$$

β_0 : intercept (baseline price)

β_1 : effect of size on price

β_2 : effect of bedrooms on price

β_3 : effect of neighborhood on price

ε : random error term

Python mirrors this split

Python makes the **goal** explicit:

Goal	Library
Inference (effects, SEs, CIs, p-values)	**statsmodels**
Prediction (fit, predict, evaluate)	**scikit-learn**

Same ideas as R; different packaging. You pick the library by what you need.

statsmodels ≈ lm (inference)

Inference-focused Python workflow: formula API via `statsmodels.formula.api` (feels like R).

```python
import statsmodels.formula.api as smf

model = smf.ols("price ~ size + bedrooms", data=df).fit()
model.summary()   # coefficients, SEs, t, p-values, R², etc.
```

Produces: Coefficients, standard errors, confidence intervals, R^2, and hypothesis tests — same role as `summary(lm(...))` in R.

Regression Output (What You Usually Report)

- **Coefficients** ($\hat{\beta}$): estimated effects of predictors
- **Standard Errors (SE)**: uncertainty of each $\hat{\beta}$
- **t-statistics**: $\hat{\beta}/\mathrm{SE}$ (signal vs noise)
- **p-values**: evidence against "no effect" ($\beta = 0$)
- $\boldsymbol{R^2}$: fraction of variance explained by the model

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}$$

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

$$\text{SST} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

Interpreting coefficients

Key interpretation (same in R and Python): Coefficients are **conditional effects** — “holding the other predictors constant.”

Example: *“Holding bedrooms constant, a one-unit increase in size is associated with a [coefficient] change in price.”*

Interpretation matches R exactly; the math is the same.

Simple vs multiple regression

- **Simple regression:** One predictor. Easier to interpret; no control for confounding.
- **Multiple regression:** Multiple predictors. Controls for confounding; coefficients are “holding other predictors constant.”

Same in R and Python; you add terms to the formula or the design matrix.

Categorical predictors

In R: Factors are handled automatically (reference category, dummies).

In Python (statsmodels formula): Use `C(neighborhood)` in the formula to treat a column as categorical. Interpretation: effects are relative to the reference category.

```
smf.ols("price ~ size + C(neighborhood)", data=df).fit()
```

scikit-learn: prediction-focused workflow

No formula. You pass numeric arrays: **X** (predictors) and **y** (outcome). Often you use a train/test split.

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X = df[["size", "bedrooms"]]
y = df["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Key difference: No inference output (no SEs, p-values). Built for fit → predict → evaluate.

Why train/test splits?

Prediction requires honest performance estimates and checking generalization to new data. So we split data, fit on train, evaluate on test.

Inference is about estimating effects and uncertainty in the model; train/test is not required (though cross-validation can still be used for model choice).

➡ For inference: use full data (or standard inference tools). For prediction: use train/test (or CV "Cross-validation).

Model evaluation

Common metrics:

- **RMSE** (root mean squared error) — prediction error in outcome units.
- **R^2** — variance explained (same idea as in R).

Important: Neither metric implies causation. Good fit ≠ causal effect.

RMSE and R^2

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- **RMSE:** lower is better (0 = perfect).
- $\boldsymbol{R^2}$**:** closer to 1 is better.

Choosing the right tool

Ask first:

1. Do I want **explanation** (effects, uncertainty) or **prediction** (accuracy)?
2. Do I need standard errors, CIs, p-values?
3. Will this model be deployed or used to score new data?

Then choose:

- **Explanation / inference** → `lm` (R) or **statsmodels** (Python).
- **Prediction / deployment** → **scikit-learn** (Python), or R's prediction-focused tools.

Active learning (15–20 min)

Explain, compare, decide

1. **Explain** — In plain language, interpret one regression coefficient (e.g. for `size`) from a model you fit.
2. **Compare** — How does `lm` (R) vs statsmodels (Python) differ in output? How does inference vs prediction differ in workflow?
3. **Decide** — Which tool would you use for (a) estimating the effect of size on price? (b) predicting price for new listings?

➡ Week 4 Notebook

Common pitfalls

- **Interpreting correlation as causation** — Regression describes association; causality requires design and assumptions.
- **Ignoring assumptions** — Linearity, independence, homoscedasticity, etc. (same in R and Python.)
- **Using R^2 alone** — High R^2 can still be misleading; think about purpose and context.
- **Forgetting the goal** — Match the tool (inference vs prediction) to the question.

Key takeaways

1. **Linear regression concepts transfer directly** from R to Python; same model, same interpretation.
2. **Python makes workflow choices explicit:** statsmodels for inference, scikit-learn for prediction.
3. **statsmodels ≈ lm** — Formula interface, coefficients, SEs, p-values, R^2.
4. **scikit-learn** — Prediction workflow: fit, predict, evaluate (train/test); no formula, no inference output.
5. **Choose the tool** that matches your goal: explanation vs prediction.

Looking ahead

Next week: Logistic regression (binary outcomes)

- Same idea: R → Python bridge; inference (statsmodels) vs prediction (scikit-learn).
- Binary outcome: logit link, classification metrics.

Bring your R→Python cheat sheet.

Thank you

Data 201 · Intro to Data Analysis in Python
Week 4 — Linear Regression: R → Python Bridge

Dr. Rebin Muhammad

Instructor note — 75 min timing (not shown to students)

Slides 1–4: ~10 min (title, welcome, agenda, learning goals, where this fits, same model different interfaces)
Slides 5–10: ~25 min (lm, Python split, statsmodels, interpretation, simple vs multiple, categoricals)
Slides 11–14: ~15 min (scikit-learn, train/test, evaluation, choosing the right tool)
Active learning: ~15–20 min (explain, compare, decide)
Wrap-up: ~5 min (pitfalls, takeaways, looking ahead)